# SE-Agent Studio 课程设计方案

## 一、项目基本信息

- 项目名称：SE-Agent Studio：面向软件工程全流程的多智能体协作开发平台
- 项目类型：软件工程课程设计 / 综合实践项目
- 项目定位：基于 CrewAI 框架与大语言模型能力，构建一个支持需求分析、任务拆解、架构设计、文档产出与测试用例生成的 Web 平台
- 适用对象：软件工程、计算机科学与技术、人工智能等相关专业学生
- 建议开发周期：6 周

## 二、项目概述

随着大语言模型和智能体系统的发展，软件开发过程正在从“单一工具辅助”逐步走向“多角色协作式智能开发”。传统课程设计项目多集中于管理系统、论坛系统、电商系统等典型 CRUD 场景，虽然实现难度适中，但在创新性、工程完整性和展示效果方面往往较为有限。

本项目拟设计并实现一个“面向软件工程全过程”的多智能体协作开发平台。平台以 CrewAI 作为智能体协作运行框架，在此基础上封装产品经理、软件架构师、后端架构师、前端开发工程师和 API 测试工程师等角色。用户只需输入自然语言形式的项目需求，系统即可按照预设流程模板自动组织不同角色完成需求理解、任务分解、系统设计、接口说明、测试用例生成与结果汇总，并将所有产物统一展示在项目空间中。

本项目的重点不是从零研发底层多智能体框架，而是面向软件工程教学场景，完成基于 CrewAI 的平台化集成、流程设计、执行追踪、结果管理和可视化展示。这样既能降低底层编排实现成本，也更符合课程设计在周期、可交付成果和答辩展示方面的要求。

## 三、研究背景与意义

### 3.1 研究背景

当前，AI 编程助手已经能够在代码补全、错误修复和文档生成等方面提供支持，但多数工具仍以“单轮问答”或“单角色助手”为主，缺乏软件工程视角下的角色分工与协作流程。现实的软件开发工作通常由产品、架构、开发、测试等多个角色共同参与，各角色关注点不同，产出形式也不同。如果能够将这些角色抽象为多个可协同工作的智能体，并将其组织为统一的软件工程工作流，将有助于提高软件开发效率，并为教学场景提供直观的实验平台。

与此同时，CrewAI 等开源智能体框架已经提供了较成熟的 Agent、Task、Crew、Flow 等机制，使开发者能够更高效地实现多智能体协作流程。因此，在课程设计场景中采用成熟框架完成平台集成，比完全自研底层编排引擎更具现实可行性，也更有助于把有限时间投入到业务流程设计、结果管理和系统展示上。

### 3.2 项目意义

本项目的研究与实现具有以下意义：

1. 体现软件工程全流程思想。项目覆盖需求、设计、测试和结果管理等多个环节，较好地符合软件工程课程的教学目标。
2. 具备较强的前沿性。相比传统信息管理系统，本项目融合了大语言模型、CrewAI 多智能体协作和软件工程流程编排，主题更贴近当前 AI 软件工程发展方向。

3. 强调工程实践能力。项目涉及前后端分离、接口设计、数据库建模、异步任务处理、结果追踪与可视化展示等内容，能够训练学生的综合开发能力。
4. 便于成果展示。系统演示时可直观体现“输入需求后自动生成多类软件工程产物”的过程，展示效果较强，适合课程答辩。

# 四、项目目标

本项目的总体目标是设计并实现一个基于 Web 的多智能体协作平台，使其能够围绕一个软件项目需求，基于 CrewAI 自动组织多个软件工程角色完成任务处理，并形成结构化成果输出。

具体目标如下：

1. 实现用户、项目、角色配置和流程模板的基础管理功能。
2. 支持用户提交项目需求，并自动生成结构化的任务拆解结果。
3. 支持基于 CrewAI 的固定协作流程模板，实现角色间的串行执行与有限并行执行。
4. 保存每次流程运行的上下文、输入输出、执行状态、耗时和异常信息，形成完整的协作记录。
5. 将需求文档、架构方案、数据库设计、接口说明、测试用例等结果统一展示与导出。
6. 提供基本的质量评审与运行统计能力，辅助用户了解协作效果。

# 五、需求分析

## 5.1 用户角色

系统主要包含以下两类用户：

1. 普通用户：可创建项目、提交需求、选择预设协作流程模板、查看智能体输出结果。
2. 管理员：可维护角色模板、流程模板、模型接入配置，查看系统运行状态与日志。

## 5.2 功能需求

### 5.2.1 用户与项目管理模块

该模块负责实现平台注册登录、用户信息维护、项目创建、项目编辑、项目列表展示和项目详情查看等功能。用户可以围绕不同的课程题目创建多个项目，并在各项目中保留对应的历史执行记录。

### 5.2.2 Agent 角色与流程模板模块

系统内置多个软件工程角色，包括产品经理、软件架构师、后端架构师、前端开发工程师、API 测试工程师等。用户可查看各角色的职责说明、输入要求、典型输出和启用状态。管理员可维护角色提示词模板、任务描述、工具绑定和流程模板配置。

### 5.2.3 需求录入与任务拆解模块

用户在项目中输入自然语言需求后，系统通过大模型对需求进行分析，提取业务目标、主要功能点、技术关键词和约束条件，并生成结构化任务清单、优先级以及建议产物目录，为后续流程执行提供输入。

### 5.2.4 基于 CrewAI 的协作编排与执行模块

平台基于 CrewAI 的 Agents、Tasks、Crews 和 Flows 机制组织协作流程。首版系统采用“固定流程模板 + 有限参数配置”的方案，支持以下两种典型方式：

1. 串行协作：前一个角色的输出作为后一个角色的输入，例如“需求分析 -> 架构设计 -> 测试设计”。
2. 有限并行协作：在某些阶段允许多个角色围绕同一需求并行生成产物，最后由系统统一汇总。

系统首版不追求可视化自由编排，而是优先保证流程稳定性、结果一致性和演示效果。

#### 5.2.5 执行追踪与上下文管理模块

系统应记录每次流程运行的输入内容、阶段状态、任务输出、执行时间、异常信息以及与项目关联的上下文数据，便于用户回溯某项结果的来源，增强系统的可解释性、可追踪性和可维护性。

#### 5.2.6 产物中心模块

系统将不同角色生成的内容进行结构化整理，形成项目产物集合，包括但不限于：

- 需求摘要
- 功能清单
- 系统架构方案
- 数据库设计建议
- API 接口文档
- 前端页面设计建议
- 测试用例与验收标准

同时，系统支持将结果导出为 Markdown 或 PDF 文档，便于课程设计报告编写和展示。

#### 5.2.7 质量评审与运行分析模块

系统可从完整性、规范性、执行耗时、角色贡献度、失败次数等维度对结果进行简单分析，并以表格和图表形式展示，为用户提供过程可视化和运行分析能力。

### 5.3 非功能需求

#### 5.3.1 性能需求

- 普通页面加载时间不超过 2 秒
- 后端普通查询接口响应时间控制在 300ms 以内
- 协作任务提交后应在 3 秒内返回任务编号和初始状态
- 长时任务需支持异步轮询或实时状态更新

#### 5.3.2 可用性需求

- 系统界面简洁直观，便于教师和学生快速上手
- 关键操作流程不超过三步，例如“创建项目 -> 输入需求 -> 启动协作”
- 主要产物展示页面支持一键查看、下载和回溯来源

#### 5.3.3 安全性需求

- 支持基本的登录认证与权限控制
- 用户数据、项目数据和调用记录应与其他用户隔离
- 大模型 API 密钥通过环境变量方式管理，不得硬编码

- 角色模板、流程模板和模型配置的修改权限仅开放给管理员

#### 5.3.4 可维护性需求

- 前后端代码结构清晰，模块划分明确
- 后端采用分层设计，并将 CrewAI 集成逻辑与业务逻辑隔离
- 数据库表结构具有一定规范性，便于维护与升级
- 流程模板和角色模板应支持后续扩展与替换

# 六、总体架构设计

## 6.1 架构选型

本项目采用“前后端分离 + 模块化单体后端 + CrewAI 运行时集成”的总体方案。前端负责用户交互与结果展示，后端负责业务逻辑、流程模板管理、任务执行编排、数据存储以及大模型接口调用。

采用该方案的主要原因如下：

1. 课程设计周期有限，基于成熟框架进行二次集成比自研编排引擎更高效。
2. 系统当前用户规模较小，模块化单体即可满足开发和展示需求，没有必要引入微服务带来的运维复杂度。
3. CrewAI 已提供较成熟的多智能体流程组织能力，项目团队可以把精力集中在教学场景适配、执行追踪和产物展示等更有价值的部分。

## 6.2 系统总体结构

```
+-------------------+        +-----------------------------+
|     Web 前端      | <----> |        FastAPI 后端          |
| Vue3 + ElementPlus|        |   REST API / 状态查询接口    |
+-------------------+        +--------------+--------------+
                                            |
                      +----------------------+----------------------+
                      |                                             |
                      v                                             v
        +---------------------------+                  +-----------------
-----+
        |        应用服务层         |                  |      数据存储层
|
        | 项目/模板/导出/统计分析    |                  |  MySQL / Redis(可选)
|
        +-------------+-------------+                  +-----------------
-----+
                      |
                      v
        +---------------------------+
        |    CrewAI 集成运行层      |
        | Agents / Tasks / Crews /  |
        | Flows / Tools / LLM 适配  |
        +-------------+-------------+
                      |
                      v
        +---------------------------+
```

```
          | OpenAI兼容接口 / DeepSeek |
          | Qwen / 本地模型服务        |
          +---------------------------+
```

### 6.3 后端模块划分

后端按照职责划分为以下几个模块：

1. 用户认证模块：处理注册、登录、身份校验与权限控制。
2. 项目管理模块：处理项目创建、项目详情、项目配置与项目历史。
3. 角色与流程模板模块：维护 Agent 配置、任务模板、流程模板和启用状态。
4. CrewAI 集成模块：负责 Agent、Task、Crew、Flow 的构建、执行与调度。
5. 执行记录模块：保存流程运行状态、阶段输出、异常信息和统计数据。
6. 结果管理模块：整理并展示各角色生成的结构化产物。
7. 导出与统计模块：支持 Markdown/PDF 导出以及运行数据统计分析。

### 6.4 前端模块划分

前端主要包含以下页面：

1. 登录注册页面
2. 项目列表页面
3. 项目详情页面
4. 需求输入与执行页面
5. 流程执行监控页面
6. 产物中心页面
7. 统计分析页面
8. 管理员配置页面

## 七、核心业务流程设计

### 7.1 项目协作主流程

1. 用户注册并登录系统。
2. 用户创建一个新的软件项目。
3. 用户选择预设流程模板，并在项目中输入自然语言需求描述。
4. 后端创建本次流程运行记录，并初始化运行状态。
5. 系统基于 CrewAI 调用需求分析角色，提取需求要点并生成结构化任务清单。
6. 系统按照预设流程依次或有限并行调用架构、开发和测试相关角色。
7. 各阶段输出结果被保存到数据库中，并实时展示在前端页面中。
8. 用户在产物中心统一查看需求文档、架构设计、接口说明和测试用例。
9. 用户可将产物导出为 Markdown 或 PDF，用于课程设计报告或答辩展示。

### 7.2 典型协作链路

建议首版系统采用以下固定协作链路：

`产品经理 -> 软件架构师 -> 后端架构师 + 前端开发工程师 -> API 测试工程师 -> 结果汇总`

其中，“后端架构师 + 前端开发工程师”阶段可根据实现情况选择并行执行或顺序执行。该协作链路能够覆盖软件工程中的关键阶段，既符合课程设计主题，也便于后续扩展更多角色。

# 八、数据库设计

## 8.1 数据库选型说明

系统选用 MySQL 作为主数据库，原因在于：

1. MySQL 安装与维护成本低，适合课程项目使用。
2. 系统数据关系清晰，用户、项目、流程模板、运行记录和产物之间存在明显的结构化关系。
3. MySQL 对中小型 Web 项目支持成熟，便于进行表设计、查询演示和数据管理。

## 8.2 核心实体设计

本系统核心实体包括：

1. 用户实体 `user`
2. 项目实体 `project`
3. Agent 配置实体 `agent_profile`
4. 流程模板实体 `workflow_template`
5. 流程运行实体 `flow_run`
6. 阶段任务运行实体 `task_run`
7. 结果产物实体 `artifact`

## 8.3 主要数据表设计

### 8.3.1 用户表 `user`

| 字段名 | 类型 | 说明 |
|---|---|---|
| id | bigint | 主键 |
| username | varchar(50) | 用户名 |
| password_hash | varchar(255) | 加密后的密码 |
| role | varchar(20) | 用户角色 |
| created_at | datetime | 创建时间 |

### 8.3.2 项目表 `project`

| 字段名 | 类型 | 说明 |
|---|---|---|
| id | bigint | 主键 |
| user_id | bigint | 所属用户 |
| name | varchar(100) | 项目名称 |
| description | text | 项目描述 |

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| status | varchar(20) | 项目状态 |
| created_at | datetime | 创建时间 |

#### 8.3.3 Agent 配置表 `agent_profile`

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| id | bigint | 主键 |
| name | varchar(100) | 角色名称 |
| code | varchar(50) | 角色编码 |
| prompt_template | text | 提示词模板 |
| model_name | varchar(100) | 默认模型 |
| enabled | tinyint | 是否启用 |

#### 8.3.4 流程模板表 `workflow_template`

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| id | bigint | 主键 |
| name | varchar(100) | 模板名称 |
| code | varchar(50) | 模板编码 |
| description | text | 模板说明 |
| config_json | longtext | 流程配置 |
| enabled | tinyint | 是否启用 |

#### 8.3.5 流程运行表 `flow_run`

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| id | bigint | 主键 |
| project_id | bigint | 所属项目 |
| workflow_template_id | bigint | 所属流程模板 |
| input_requirement | longtext | 原始需求输入 |
| status | varchar(20) | 执行状态 |
| state_json | longtext | 流程状态快照 |
| started_at | datetime | 开始时间 |

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| finished_at | datetime | 结束时间 |

#### 8.3.6 阶段任务运行表 `task_run`

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| id | bigint | 主键 |
| flow_run_id | bigint | 所属流程运行 |
| agent_code | varchar(50) | 执行角色编码 |
| task_name | varchar(100) | 任务名称 |
| input_text | longtext | 输入内容 |
| output_text | longtext | 输出内容 |
| status | varchar(20) | 执行状态 |
| token_usage | int | Token 消耗 |
| error_message | text | 错误信息 |
| started_at | datetime | 开始时间 |
| finished_at | datetime | 结束时间 |

#### 8.3.7 产物表 `artifact`

| 字段名 | 类型 | 说明 |
| --- | --- | --- |
| id | bigint | 主键 |
| project_id | bigint | 所属项目 |
| flow_run_id | bigint | 来源流程运行 |
| type | varchar(50) | 产物类型 |
| title | varchar(200) | 产物标题 |
| content | longtext | 产物内容 |
| created_at | datetime | 创建时间 |

### 8.4 表关系说明

- 一个用户可以拥有多个项目
- 一个项目可以关联多个流程运行记录
- 一个流程模板可以被多个项目重复使用
- 一次流程运行可以包含多个阶段任务运行记录
- 一个项目可以生成多个产物
- 一个角色配置可以参与多个阶段任务执行

# 九、技术栈设计

## 9.1 前端技术

- Vue 3：构建用户界面
- TypeScript：提高代码可维护性与规范性
- Vite：提升前端开发与构建效率
- Pinia：管理全局状态
- Element Plus：快速搭建后台管理类界面
- ECharts：实现统计分析页面中的图表展示

## 9.2 后端技术

- FastAPI：构建高性能 RESTful API
- CrewAI：实现多智能体角色定义、任务组织和流程执行
- SQLAlchemy：实现 ORM 映射
- Pydantic：进行数据校验与结构化输出约束
- Redis（可选）：缓存执行状态与短期任务数据
- BackgroundTasks 或 Celery（可选）：实现异步任务执行与结果更新

## 9.3 数据与部署技术

- MySQL：持久化存储结构化数据
- Docker：统一开发与部署环境
- Nginx：部署前端静态资源并反向代理后端接口

## 9.4 AI 能力接入

系统通过统一的模型调用接口接入大语言模型服务，可兼容 OpenAI 格式接口，也可根据环境接入 DeepSeek、Qwen 或本地模型服务。后端基于 CrewAI 将不同角色封装为 Agent，将结构化产出要求封装为 Task，将固定协作模板封装为 Crew 或 Flow，并结合项目上下文完成流程执行。

# 十、关键设计决策与理由

## 10.1 采用 CrewAI 作为多智能体协作运行框架

首版系统不自研底层多智能体编排引擎，而是采用 CrewAI 提供的 Agent、Task、Crew、Flow 能力完成流程组织。该选择能够显著降低底层实现复杂度，使团队把更多精力投入到软件工程场景设计、数据管理与结果展示上。

## 10.2 采用前后端分离 + 模块化单体后端

前后端分离便于多人协作开发，也更符合当前主流 Web 项目开发模式。后端采用模块化单体架构，能够在保证结构清晰的前提下降低部署和调试复杂度，适合课程设计周期与团队规模。

## 10.3 首版采用固定流程模板

为控制项目复杂度，首版系统不实现完全自由的流程编排，而是预设一条典型的软件工程协作链路，并允许对模型、角色启用状态和少量参数进行配置。这样既能体现多智能体协作思想，又可确保系统稳定可演示。

## 10.4 以结构化文档产物和过程追踪为核心输出

课程设计项目更强调“过程合理”和“结果可展示”。因此本系统优先保证需求文档、架构方案、接口设计、测试用例和执行记录等内容的生成与展示质量，而不是追求复杂的自动代码生成能力。

# 十一、项目实施计划

## 第 1 周：需求分析与方案修订

- 明确项目目标、边界与演示场景
- 结合 CrewAI 完成功能需求与非功能需求分析
- 绘制系统总体架构图和功能模块图
- 设计数据库表结构和流程模板草案

## 第 2 周：基础框架与运行骨架搭建

- 初始化前后端工程
- 配置数据库连接与基础表结构
- 完成 FastAPI 与 CrewAI 的基础集成骨架
- 完成登录注册和项目管理模块

## 第 3 周：角色模板与需求分析功能开发

- 完成 Agent 角色配置页面
- 完成流程模板配置页面
- 实现需求输入、结构化解析与任务拆解模块

## 第 4 周：流程执行与运行记录开发

- 实现固定流程模板下的多 Agent 协作执行
- 实现执行状态展示、阶段日志保存和异常记录
- 完成基础的结果汇总逻辑

## 第 5 周：产物中心与统计分析开发

- 实现需求文档、架构文档、测试文档的集中展示
- 实现 Markdown/PDF 导出功能
- 实现基础统计图表与质量评审页面

## 第 6 周：测试、优化与总结

- 完成接口测试与页面联调
- 修复缺陷并优化交互体验
- 编写课程设计报告、系统说明书与答辩 PPT

# 十二、预期成果

本项目完成后，预期形成以下成果：

1. 一个可正常运行的基于 CrewAI 的 Web 平台系统
2. 一套完整的软件工程课程设计文档
3. 一份系统演示视频或答辩演示材料

4. 一套软件项目需求输入后自动生成的示例产物

# 十三、项目创新点

1. 面向软件工程教学场景设计多角色协作流程，而非停留在通用问答助手层面。
2. 基于 CrewAI 快速构建“需求分析 -> 架构设计 -> 测试产出”的一体化流程，突出工程落地而非重复造轮子。
3. 将结构化输出、执行追踪、产物管理和可视化展示结合起来，提升系统的教学价值和答辩展示效果。
4. 相较于传统管理系统类课设，更贴近当前 AI 软件工程的发展趋势。

# 十四、可行性分析

## 14.1 技术可行性

本项目使用的技术栈均为成熟技术，前后端开发门槛适中，网上资料丰富，适合作为课程设计项目实施。CrewAI 已提供较成熟的多智能体角色组织和流程执行能力，能够降低底层编排开发难度。项目的主要技术挑战将集中在提示词设计、结构化输出约束、流程状态管理和前端展示联动等方面，整体可控。

## 14.2 时间可行性

项目首版功能聚焦于“项目管理 + 基于 CrewAI 的固定流程协作 + 结果展示”，需求范围适中。若由 2 至 3 名学生协作开发，在 6 周左右具备较高完成可能性。

## 14.3 经济可行性

项目开发所需软件环境基本免费。若使用云端大模型接口，成本主要来自测试阶段的接口调用费用；若使用本地模型或校园环境中的兼容接口，整体成本可进一步降低。

# 十五、总结

SE-Agent Studio 课程设计项目以“CrewAI 多智能体协作 + 软件工程流程”为核心，兼顾创新性、工程性与可展示性。该项目不仅能够体现软件工程专业学生在需求分析、系统设计、数据库设计、前后端开发和测试验证等方面的综合能力，还能够结合当前 AI 技术发展趋势，形成较有特色的课程设计成果。

从项目复杂度、技术实现难度和答辩展示效果来看，该题目具有较高的课程设计适配度。通过采用成熟的 CrewAI 框架，项目团队可以减少底层重复开发，把重点放在平台设计、流程组织、产物管理与教学展示上，从而更稳妥地在课程周期内完成高质量实现。